UNIT-4

1. Explain the purpose of Normalization and discuss 1NF, 2NF with suitable examples.
2. Explain 4NF, and 5NF with suitable examples.
3. Explain Functional Dependency and different types of dependencies with examples.
4. Compare 3NF and BCNF with suitable examples.
5. Explain Lossless Join and Dependency Preserving Decomposition.
6. Explain Transaction Management and ACID properties with examples.
7. Explain Transaction States with a neat state transition diagram.
8. Explain Concurrent Execution and Serializability with examples.
9. Discuss conflict serializability and view serializability with suitable examples.
10. Explain Recoverability and Recovery Algorithms in DBMS.

Q. Explain the purpose of Normalization and discuss 1NF, 2NF with suitable examples. (10m)

Ans: 1. **Purpose of Normalization :**

Normalization is the process of organizing data in a database to reduce data redundancy and avoid data inconsistency. It helps to :

- Eliminate duplicate data.
- Improve data integrity and accuracy.
- Make the database easier to maintain and update.
- Reduce storage space.
- Avoid problems during insertion, deletion and modification of data.

2. **1NF (First Normal Form) :**

A table is in 1NF if all the attributes contain atomic values (single values), no repeating groups and no multi-valued attributes.

Example :

Consider a table "Student" which is not in 1NF (because Subjects contains multiple values).

Student (Not in 1NF)

RollNo	Name	Dept	Subjects
101	Ravi	CSE	DS, DBMS
102	Sita	ECE	OS, CN
103	Hari	ME	FEA, CAD

Here, each cell contains a single value.

⇒ Convert it to 1NF by splitting the multi-valued attribute into separate rows.

Student (1NF)

RollNo	Name	Dept	Subject
101	Ravi	CSE	DS
101	Ravi	CSE	DBMS
102	Sita	ECE	OS
102	Sita	ECE	CN
103	Hari	ME	FEA
103	Hari	ME	CAD

3. **2NF (Second Normal Form) :**

A table is in 2NF if it is in 1NF and there is no partial dependency of non-key attributes on a candidate key. In other words, all non-key attributes should be fully functionally dependent on the primary key.

Example :

Consider a table "Orders" where some attributes depend only on part of the composite key (OrderID, ProductID).

Orders (Not in 2NF)

OrderID	ProductID	ProductName	UnitPrice	Quantity	Total
0001	P01	Laptop	50000	2	100000
0001	P02	Mouse	500	1	500
0002	P03	Keyboard	1500	1	1500

Here, ProductName and UnitPrice depend only on ProductID, not on the whole key (OrderID, ProductID). So it is not in 2NF.

Convert it to 2NF :

Orders (2NF)

OrderID	ProductID	Quantity	Total
0001	P01	2	100000
0001	P02	1	500
0002	P03	1	1500

Product (separate table)

ProductID	ProductName	UnitPrice
P01	Laptop	50000
P02	Mouse	500
P03	Keyboard	1500

By separating the dependent attributes, we remove partial dependency and achieve 2NF.

Q. Explain 4NF, and 5NF with suitable examples. (10m)

1. **4NF (Fourth Normal Form):**

4NF is a higher form of normalization. A table is in 4NF if it is in 3NF and has no multi-valued dependencies (MVD) on a non-key attribute. In other words, a non-key attribute should not determine multiple values independently of another non-key attribute.

Purpose:

- Removes multi-valued dependencies.
- Prevents redundancy and update anomalies.
- Ensures each fact is stored only once.

Example:

Consider a table "Student_Skills" where a student can have multiple skills and multiple hobbies. Storing both skills and hobbies in the same table creates a multi-valued dependency.

Student_Skills (Not in 4NF)

StudentID	StudentName	Skill	Hobby
101	Ravi	Python	Cricket
101	Ravi	Python	Music
101	Ravi	Java	Cricket
102	Sita	C++	Reading

Here, Skill and Hobby are independent multi-valued attributes for the same student. This creates a multi-valued dependency.

To convert it to 4NF, we separate the multi-valued attributes into different tables:

Student

StudentID	StudentName
101	Ravi
102	Sita

Student_Skill

StudentID	Skill
101	Python
101	Java
102	C++

Student_Hobby

StudentID	Hobby
101	Cricket
101	Music
102	Reading

Now each non-key attribute depends only on the key and there is no MVD. So, the table is in 4NF.

2. **5NF (Fifth Normal Form):**

5NF is a higher normal form. A table is in 5NF if it is in 4NF and has no join dependencies. A join dependency means that a table can be obtained by joining two or more tables without any loss of information.

Purpose:

- Removes join dependencies.
- Avoids redundancy and data inconsistency.
- Ensures the data can be reconstructed from smaller tables without loss.

Example:

Consider a relation "Project_Employee" where a project can have multiple employees and an employee can work on multiple projects. Storing it in one table creates a join dependency.

Project_Employee (Not in 5NF)

ProjectID	ProjectName	EmpID	EmpName
P1	Website	E1	Ravi
P1	Website	E2	Sita
P2	Mobile	E2	Sita
P2	Mobile	E3	Hari

This table can be obtained by joining Project, Employee and Project_Employee tables. So it has a join dependency and is not in 5NF.

To make it in 5NF, we create three separate tables:

Project

ProjectID	ProjectName
P1	Website
P2	Mobile

Employee

EmpID	EmpName
E1	Ravi
E2	Sita
E3	Hari

Project_Employee

ProjectID	EmpID
P1	E1
P1	E2
P2	E2
P2	E3

Now the relation is in 5NF because it can be reconstructed by joining the tables without losing any information.

Q. Explain Functional Dependency and different types of dependencies with examples. (10m)

Ans: **1. Functional Dependency (FD):**

A functional dependency is a relationship between two sets of attributes in a relation. We say that attribute Y is functionally dependent on attribute X (if denoted as $X \rightarrow Y$) if each value of X is associated with exactly one value of Y.

In simple words, if we can find the value of Y, then we can uniquely find the value of X.

Example: Consider a relation "Student" with attributes (RollNo, Name, Dept). Here, RollNo → Name and RollNo → Dept, because each roll number uniquely identifies one student and one department.

Student

RollNo	Name	Dept
101	Ravi	CSE
102	Sita	ECE
103	Hari	ME

So, RollNo → Name and RollNo → Dept are functional dependencies.

2. Types of Functional Dependencies:

There are different types of functional dependencies. They are:

2.1 Trivial Functional Dependency:

If Y is a subset of X, then $X \rightarrow Y$ is called a trivial FD.

Example: If we have (A, B) → A, then it is a trivial FD. (Because A is already part of the left side.)

2.2 Non-Trivial Functional Dependency:

If Y is not a subset of X, then $X \rightarrow Y$ is called a non-trivial FD.

Example: RollNo → Name is a non-trivial FD (as Name is not part of RollNo).

2.3 Complete Functional Dependency:

A functional dependency $X \rightarrow Y$ is called complete if no proper subset of X can determine Y.

Example: (StudentID, CourseID) → Grade is a complete FD, because neither StudentID → Grade nor CourseID → Grade works alone.

2.4 Transitive Functional Dependency:

If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$ is called a transitive FD.

Example: StudentID → DeptID and DeptID → DeptName

So, StudentID → DeptName is a transitive dependency.

3. Other Types of Dependencies:

There are some other dependencies which are also important in relational databases.

3.1 Multivalued Dependency (MVD):

If one attribute can have multiple values for a single value of another attribute, it is called a multivalued dependency.

Example: Student → Hobby (a student can have multiple hobbies).

3.2 Join Dependency (JD):

A relation R has a join dependency if it can be reconstructed from two or more smaller relations by joining them.

Example: R(A, B, C) can be decomposed into $R_1(A, B)$ and $R_2(A, C)$.

Conclusion: Functional dependency helps to reduce redundancy and maintain data integrity. Different types of dependencies (trivial, non-trivial, complete, transitive, MVD and JD) are used to design a good database schema.

Q. Compare 3NF and BCNF with suitable examples. (10m)

Ans:

1. **Introduction :**

3NF (Third Normal Form) and BCNF (Boyce-Codd Normal Form) are both higher normal forms in database normalization. They are used to remove redundancy and avoid update anomalies. BCNF is stricter than 3NF.

2. **3NF (Third Normal Form) :**

- A table is in 3NF if it is in 2NF and there is no transitive dependency between non-key attributes.
- In 3NF, every non-key attribute must depend only on the candidate key, not on another non-key attribute.
- It removes transitive dependency.

Example :

Consider a table "Student" with a transitive dependency (DepartmentName depends on DeptID and DeptID depends on StudentID).

Student (Not in 3NF)

StudentID	StudentName	DeptID	DepartmentName
101	Ravi	D1	CSE
102	Sita	D2	ECE
103	Hari	D3	ME

Here, DepartmentName depends on DeptID, and DeptID depends on StudentID. So, it is not in 3NF.

To make it in 3NF, we split it into two tables:

Student (3NF)

StudentID	StudentName	DeptID
101	Ravi	D1
102	Sita	D2
103	Hari	D3

Department (3NF)

DeptID	DepartmentName
D1	CSE
D2	ECE
D3	ME

Now, there is no transitive dependency in any table. So, both tables are in 3NF.

3. **BCNF (Boyce-Codd Normal Form) :**

- A table is in BCNF if for every non-trivial functional dependency $X \rightarrow Y$, X must be a super key.
- It is stricter than 3NF.
- In BCNF, there should be no determinant that is not a candidate key.

Example :

Consider a table "Course" where CourseID determines CourseName, and also InstructorID determines InstructorName (which is not a candidate key).

Course (In 3NF but not in BCNF)

CourseID	CourseName	InstructorID	InstructorName
C101	DBMS	I1	Ramesh
C102	OS	I2	Suresh
C103	Networks	I3	Lakshmi

Here, InstructorID → InstructorName is a functional dependency, but InstructorID is not a candidate key. So, it is not in BCNF even though it is in 3NF.

To make it in BCNF, we decompose it into two tables:

Course (BCNF)

CourseID	CourseName	InstructorID
C101	DBMS	I1
C102	OS	I2
C103	Networks	I3

Instructor (BCNF)

InstructorID	InstructorName
I1	Ramesh
I2	Suresh
I3	Lakshmi

Now, in both tables, the determinant is a candidate key. So, both are in BCNF.

4. **Conclusion :** 3NF removes transitive dependency, while BCNF removes all non-trivial dependencies where the determinant is not a candidate key. BCNF is stricter and ensures better data integrity.

Q. Explain Lossless Join and Dependency Preserving Decomposition.

(10 marks)

Ans: **1) Lossless Join Decomposition :**

Definition : A decomposition of a relation R into two or more relations is called lossless join decomposition if the original relation can be obtained exactly by joining the decomposed relations, without any loss of information.

In simple words : The natural join of the decomposed relations should give back the same data as the original relation.

Why it is important :

- Prevents loss of data.
- Maintains the correctness of the database.
- Ensures that no spurious tuples are produced after the join.

Example : Suppose a relation $R(A, B, C)$ is decomposed into $R_1(A, B)$ and $R_2(B, C)$. If the common attribute B is a key in one of the relations (or the decomposition satisfies the proper condition), then the join of R_1 and R_2 will give back the original relation R. Hence, it is a lossless join decomposition.

2) Dependency Preserving Decomposition :

Definition : A decomposition of a relation R is called dependency preserving if all functional dependencies (FDs) that hold in R are also preserved in the decomposed relations.

In simple words : We should be able to enforce the same constraints (functional dependencies) on the smaller relations, without having to join them.

Why it is important :

- Maintains the integrity constraints.
- Reduces the complexity of checking dependencies.
- Helps in efficient query processing and updates.

Example : If $R(A, B, C)$ has FDs $\{A \rightarrow B,\ B \rightarrow C\}$, then after decomposition, these FDs should be exactly present in the smaller relations (either in R_1 or R_2 or both), so that we can enforce them without joining.

Conclusion : Lossless join decomposition ensures no loss of data, and dependency preserving decomposition ensures that all functional dependencies are maintained in the decomposed relations.

Q. Explain Transaction Management and ACID properties with examples.

(10 marks)

Ans: **1) Transaction Management :**

Transaction management is the process of handling a group of related database operations as a single unit. It ensures that the data is kept consistent and correct, even when multiple users are working on the database at the same time.

Main activities in transaction management :

- Begin Transaction - starts a new transaction.
- Commit - makes the changes permanent.
- Rollback - undoes the changes if something goes wrong.
- Concurrency control - manages access when many users work together.
- Recovery management - brings the database back to a correct state after a failure.

Example : Suppose a bank transfers money from one account to another. Both the debit and credit operations are treated as one transaction. If any problem occurs, the system will rollback (undo) the changes. If everything is fine, it will commit (save) the changes.

2) ACID Properties :

ACID is an acronym for Atomicity, Consistency, Isolation and Durability. These properties ensure that transactions are processed correctly and safely.

1. **Atomicity :** Either all the operations in a transaction are completed, or none of them are. No partial changes are allowed.

   Example : In a fund transfer, either money is debited from A and credited to B, or no money is transferred at all.

2. **Consistency :** A transaction takes the database from one valid state to another valid state, according to the rules (constraints).

   Example : If the balance should always be positive, a transaction will not allow a negative balance.

3. **Isolation :** Transactions are independent of each other. The result of one transactions should not affect another transactions running at the same time.

   Example : If two users are updating the same record, isolation ensures that they do not read or write inconsistent data.

4. **Durability :** Once a transaction is committed, the changes are permanent, even if there is a power failure or system crash.

   Example : After a bank transfer is completed and committed, the money will remain in the new account, even if the system crashes.

Conclusion : Transaction management handles the execution of transactions and ACID properties ensure that the transactions are processed correctly, safely and consistently.

Q. Explain Transaction States with a neat state transition diagram. (10 marks)

Ans:

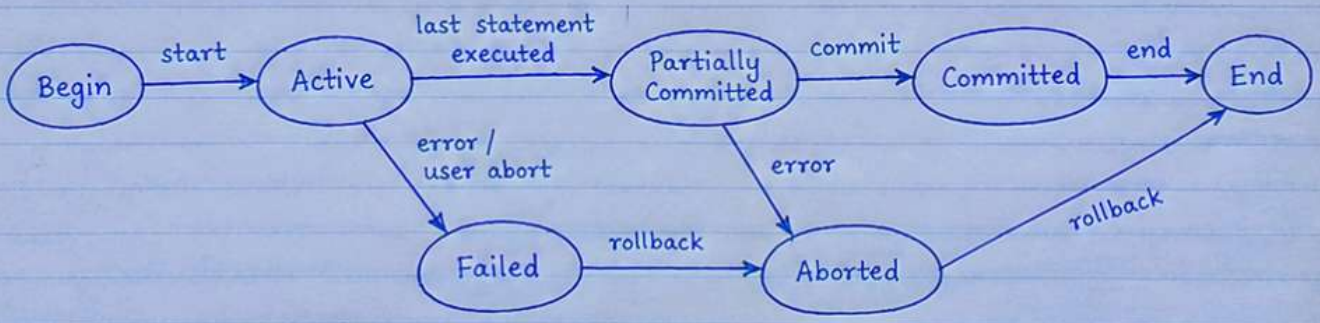
1) Transaction States :

A transaction goes through a number of states during its life cycle. These states show the current condition of the transaction and the actions taken by the system. The state transition diagram shows how a transaction moves from one state to another.

2) States of a Transaction :

1. Active : The transaction is executing its read/write operations.
2. Partially Committed : The last statement has been executed, but the commit is not yet done.
3. Committed : The transaction is successfully completed and all changes are saved in the database.
4. Failed : The transaction is terminated due to some error or user request.
5. Aborted : The transaction is rolled back and all changes are undone.

3) State Transition Diagram :



4) Explanation of Transitions :

1. Begin → Active : When the transaction starts, it goes to Active state.
2. Active → Partially Committed : When the last statement is executed, it moves to Partially Committed state.
3. Partially Committed → Committed : If commit is successful, it moves to Committed state.
4. Any state → Failed : If an error occurs, the transaction goes to Failed state.
5. Failed → Aborted : The system rolls back the changes and moves to Aborted state.
6. Aborted → End : After rollback, the transaction ends.
7. Committed → End : After completion, the transaction ends.

5) Conclusion :

The state transition diagram helps to understand the life cycle of a transaction and the different states it goes through during execution.

Q. Explain Concurrent Execution and Serializability with examples. (10 marks)

Ans:

1) Concurrent Execution :

Definition : Concurrent execution means running two or more transactions at the same time. The operations of different transactions are interleaved (mixed) in time. It improves CPU / system utilization and gives better performance.

Why it is important :

- Increases system throughput.
- Reduces waiting time of users.
- Makes better use of resources.

Example : Consider two transactions T_1 and T_2.

T_1 (Transfer ₹100 from A to B)	T_2 (Check balance of A)
1. read(A) 2. A = A − 100 3. write(A) 4. read(B) 5. B = B + 100 6. write(B)	1. read(A) 2. print(A) 3. if (A >= 0) then print("Balance is sufficient") else print("Insufficient balance")

The operations of T_1 and T_2 are mixed (interleaved). This is concurrent execution.

2) Serializability :

Definition : A schedule is called serializable if its result is the same as some serial (one after another) execution of the same transactions. It means the concurrent execution does not change the final result.

Types : 1. Conflict Serializability 2. View Serializability

Example : Let two transactions T_1 and T_2 be executed concurrently.

T_1 : read(X) X = X + 10 write(X)

T_2 : read(X) X = X − 5 write(X)

If the schedule is : T_1 : r(X) w(X) T_2 : r(X) w(X) (interleaved)

The final value of X will be the same as the serial order $T_1 \rightarrow T_2$.

So, this schedule is serializable.

Conclusion : Concurrent execution improves performance, but it is correct only if the schedule is serializable.

Q. Discuss Conflict Serializability and View Serializability with suitable examples. (10 marks)

Ans: 1) **Conflict Serializability :**

Definition : A schedule is said to be conflict serializable if it can be obtained from a serial schedule by swapping non-conflicting operations. In other words, the order of conflicting operations must be the same as in some serial schedule.

Types of conflicts :

- Read - Write (RW) : one transaction reads and another writes.
- Write - Read (WR) : one transaction writes and another reads.
- Write - Write (WW) : two transactions both write.

Example : Consider two transactions T_1 and T_2.

T_1 : R(A) W(A) R(B) W(B)

T_2 : R(A) W(A) R(B) W(B)

A possible interleaved schedule :

Step	1	2	3	4	5	6	7	8	9
T_1	R(A)	W(A)	R(B)	W(B)	-	-	-	-	-
T_2	-	-	-	-	R(A)	W(A)	-	R(B)	W(B)

The conflicting operations are in the same order as T_1 and T_2 in a serial schedule (T_1 followed by T_2). Hence, this schedule is conflict serializable.

2) **View Serializability :**

Definition : A schedule is said to be view serializable if it is view equivalent to some serial schedule. Two schedules are view equivalent if for every data item, the reads and final writes are from the same transactions.

Example : Consider two transactions T_1 and T_2.

T_1 : R(A) W(A) T_2 : R(A) W(A)

Interleaved schedule :

Step	1	2	3	4	5	6	7	8
T_1	R(A)	W(A)	-	-	-	-	-	-
T_2	-	-	R(A)	W(A)	-	-	-	-

In this schedule, T_2 reads the value written by T_1, and the final write to A is also by T_2. So, it is view equivalent to the serial schedule T_1 followed by T_2. Hence, it is view serializable.

Conclusion : Conflict serializability is easier to check, while view serializability is more general. Every conflict serializable schedule is view serializable, but not every view serializable schedule is conflict serializable.

Q. Explain Recoverability and Recovery Algorithms in DBMS. (10 marks)

Ans:

1) Recoverability :

Definition : Recoverability means that if a transaction T_j reads a value written by transaction T_1, then T_j can commit only after T_1 has committed. If T_1 aborts, then T_j must also abort.

Why it is important ?

- It prevents the system from getting inconsistent data.
- It ensures that the database remains in a correct state.

Example : T_1 writes a value in an item and commits. T_2 reads this value and then commits. This is recoverable. But if T_2 reads a value written by T_1 and commits before T_1, it is not recoverable.

2) Recovery Algorithms :

Recovery algorithms are methods used to bring the database back to a consistent state after a failure (such as system crash or transaction abort). These algorithms use a log of all actions (write, commit, abort).

Main Recovery Algorithms :

1) **Deferred Update (No-Undo) :**
   - Updates are written to the database only after the transaction commits.
   - If a failure occurs, no rollback is needed.
   - **Example :** In a banking system, the changes are written after commit.

2) **Immediate Update (Undo/Redo) :**
   - Updates are written to the database during the transaction.
   - If a failure occurs, the system performs undo for uncommitted transactions and redo for committed ones.
   - **Example :** Many real-time systems use this method.

3) **Log Based Recovery :**
   - The system maintains a log file of all operations.
   - During recovery, the log is used to undo or redo the necessary transactions.
   - **Example :** Write-Ahead Log (WAL) technique.

Conclusion : Recoverability ensures that transactions follow the proper order and Recovery algorithms help to restore the database to a consistent state after a failure.